

4.2.4 Source Masking

The final step in the RM program is to mask the source once all calculations for that source has been completed. I added this step to the Python script after repeated failures of correctly fitting a Gaussian to the second source in a pair. The program was either fitting the first source twice or fitting both sources to a single Gaussian. An example of each is shown in Figure 4.10. After comparing multiple circular masks via manual inspection, I found that using an ellipse with a major axis of $1.75 \text{ FWHM}_{\text{major}}$ and a minor axis of $1.75 \text{ FWHM}_{\text{minor}}$ was the optimal size mask to entirely block out the signal of the first source, while not blocking any significant signal from the second. This addition significantly reduced the number of fitting errors, but unfortunately did not resolve every instance of one Gaussian being fit to both sources.

There were nine pairs where both sources were fit to one Gaussian when examining the first source in the pair, so the solution stated above did not resolve this issue. These pairs were recorded, and the RM was updated to include a 3×3 pixel mask placed over the second source in the pair when fitting the first source. This mask resolved the issue of fitting both sources to a single Gaussian for seven of the nine pairs. One pair (G3) required a 5×5 pixel mask instead of the standard mask, and another pair (U2) required a 9×9 pixel mask. Figure 4.11 shows the fitted Gaussian to the first source in a pair (G15) before and after the mask was added. This feature can be enabled by a future user if desired.

Two pairs (G10 and G28) were triple sources, where not only are the lobes visible in PI, but their cores were as well. These pairs required an additional mask for the core before fitting the second lobe. These two pairs were the only pairs that required an additional mask for correct fitting.

4.2.5 Additional Modifications made to the Python RM Program

The first version of the RM program in Python had multiple software bugs that needed to be fixed. Without debugging the program, it would have given inaccurate RMs, rendering the

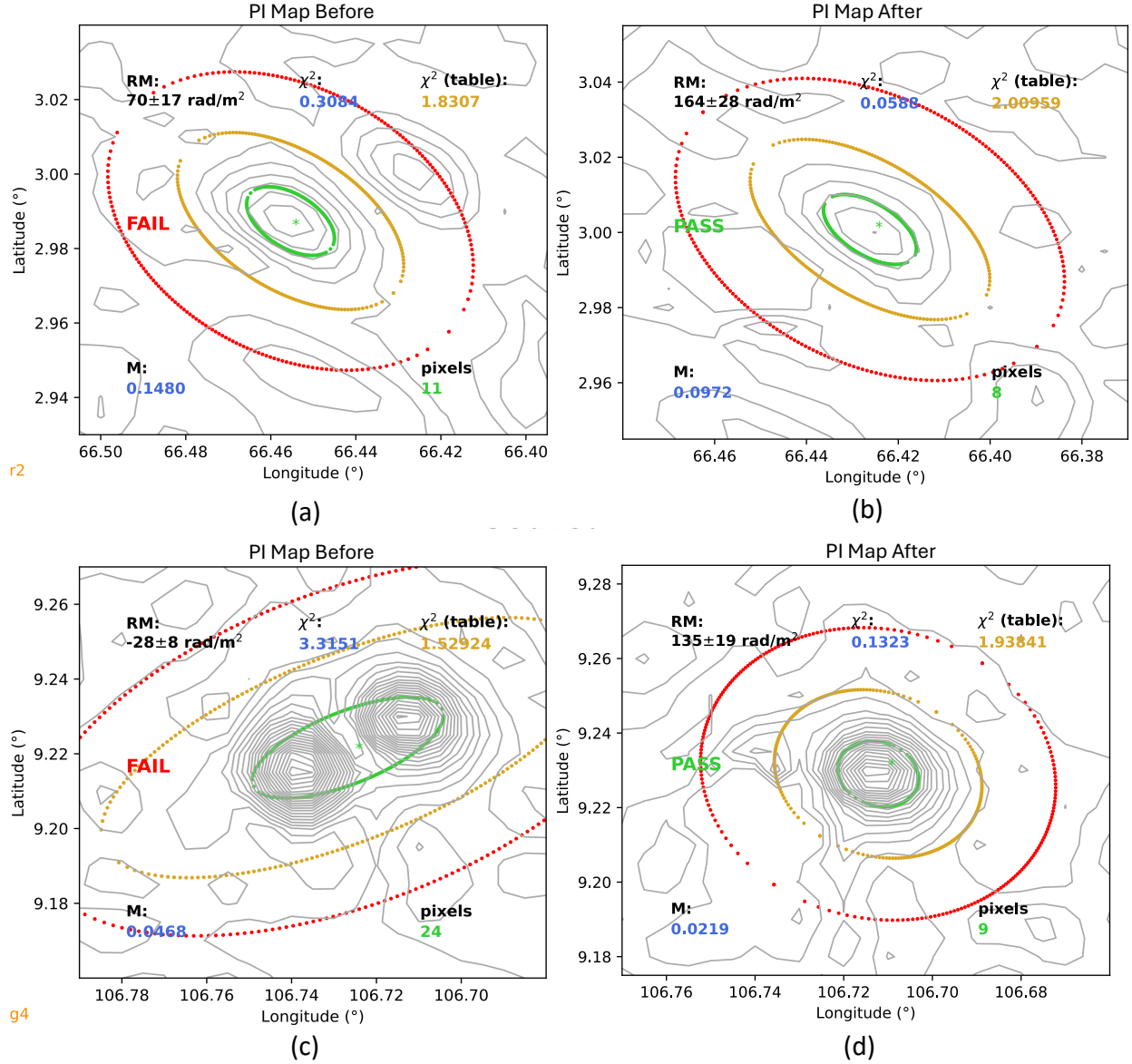


Figure 4.10: Gaussian Fitting Failures and Fixes.

These figures demonstrate why the masking needed to be added. Panels (a) and (c) show examples before masking was added, and Panels (b) and (d) show the fitting with masking. Panels (a) and (b) show an example of a source that the program fit twice, and Panels (c) and (d) show the program fitting one Gaussian to both sources. In green, the FWHM is shown, and the inner and outer edges of the background removal ring are shown in yellow and red, respectively. In the initial program plots, the mosaic name was printed in the bottom left hand corner of the PI plot.

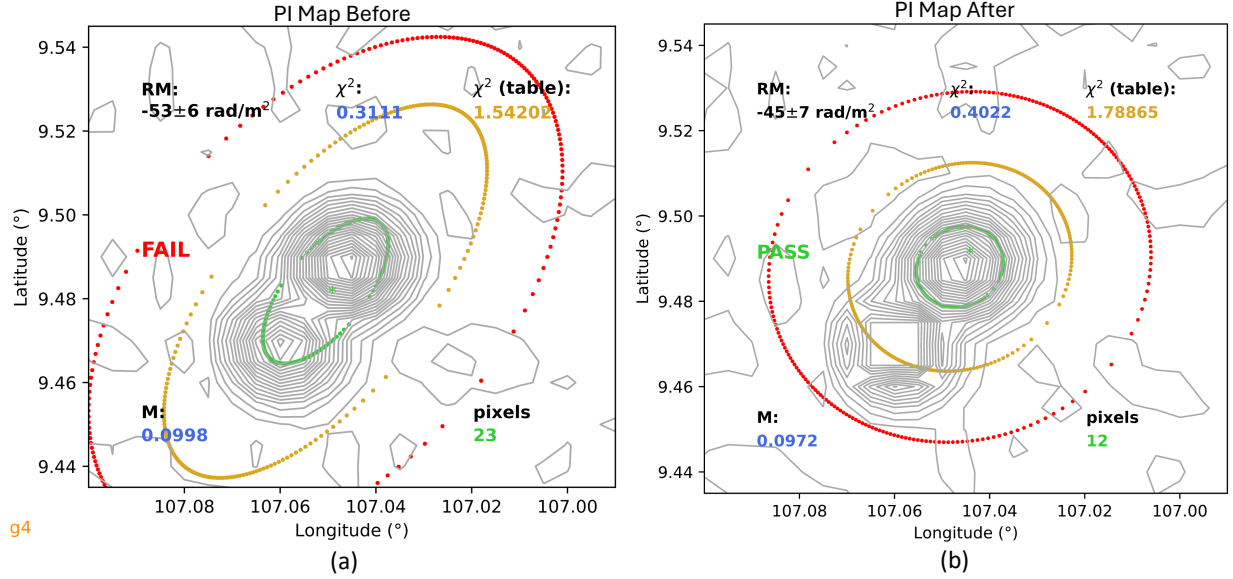


Figure 4.11: Second Source Mask.

This figure shows an example of where the second source needed to be masked before the first source was fit. Panel (a) the initial fit for the first source before the mask was added, and Panel (b) is the fit after the square mask was added. In green the FWHM is shown, and the inner and outer edges of the background removal ring is shown in yellow and red, respectively. In the initial program plots, the mosaic name was printed in the bottom left hand corner of the PI plot.

Python version of the program unusable for my project and future projects. The changes made are documented below.

Arctan function modification

Occasionally the polarisation angles would be miscalculated. Equation (2.14) gives the polarisation angle using the arctan function. In Python, there are two primary arctan functions. The first is `np.arctan`, which takes a single parameter and thus can only return an angle between $-\pi/2$ and $\pi/2$. This function was originally used to calculate the polarisation angles, and was the cause of the incorrect polarisation angles. I changed the program to use the second common arctan function, `np.arctan2` which takes in numerator and denominator separately to calculate the angle between $-\pi$ and π . Once this change was made, the polarisation angles and the majority of the RMs of the sources matched the IDL output.

Array copying issue

The array containing the RM values was not copied correctly. In Python, objects are “pointers”. Therefore, unless you explicitly make a copy of an object, the variables containing the objects are linked together because the variables are pointing to the same object in the computer’s memory. For example, if you set $y = x$, and you change y , it also changes x . To avoid this issue, one must copy the original object to create a separate object. In the RM code, the RM map colormap has a min of -500 rad/m^2 and a max of 500 rad/m^2 . In IDL, all values in the data array must fall within the colormap range, unlike in Python, where values outside the colormap range are set to one extreme of the colormap. The Python program was created to be as close of a match as possible to the IDL version, and hence it also sets limits on the RM array used to plot the RM map. However, during this process, the RM array was not copied, and the RM values were later used to plot the line of best fit on the peak pixel linear fit plot. For sources with RMs that fell outside the colormap range, this caused the RM of the peak pixel and the overall RM of the source to be calculated incorrectly. The bug was corrected by copying the RM array.

Error bars on peak pixel linear fit plot

The error bars in the plot were too large because the error bar functions in IDL and Python interpret their input value differently. In IDL, the input number is interpreted as the full length of the error bar. Therefore, in the IDL code, the input value for the error bars was two times the uncertainty in the polarisation angle. However, the Python function interpreted the input value to be the length from the central point to the end of the error bar. This resulted in the error bars being double their length in the original translation.

Minimum Uncertainty in the Polarisation Angle

The minimum uncertainty in the polarisation angle needed to be set to 1° , as the ST does not have the sensitivity to determine polarisation angles that precisely. Without it, one of

the sources, G13a, failed when it should have passed. The peak pixel plot shows clear linear behaviour upon visual inspection; however, the probability of fit of the source was still below 10% due to the small uncertainty in the polarisation angle across all 4 bands. The peak pixel plot is shown in Figure 4.12a. The inclusion of the minimum uncertainty for all four bands increased the probability of fit and allowed the source to pass. The new peak pixel linear fit plot is shown in Figure 4.12b.

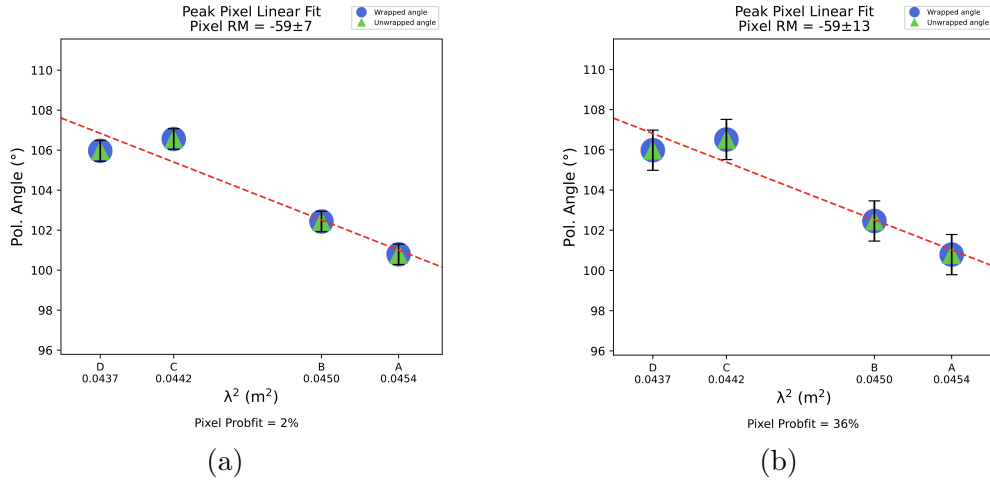


Figure 4.12: The bright PI source (G13a) with and without a minimum uncertainty in PA. The left panel shows the peak pixel linear fit graph of the source (G13a) without a minimum uncertainty in PA, which fails the probability of fit test despite displaying linear behaviour. The right panel shows the peak pixel linear fit with a minimum uncertainty of 1° in the τ and the source (G13a) passes the probability of fit test.

Plotting unwrapped angles

Initially, the unwrapped angles were plotted. In the original IDL code by Dr. Brown, the wrapped angles were plotted using blue circles and the unwrapped angles were displayed using the green triangles. However, in the first iteration of the Python code only the unwrapped angles were being plotted, shown in Figure 4.13a. The code is now updated to plot both the wrapped and unwrapped angles using the same symbols, and a legend was later added to clarify what the two symbols represent. The band that was used to determine where to start wrapping was also switched from band D to band A to match the original code.

A single line of best fit

Once the wrapped angles were added, it revealed another issue. The line of best fit was being plotted multiple times. One line had the intrinsic polarisation angle, τ_0 , as the intercept, the second one was had the intercept at $\tau_0 - 180^\circ$, and the third one had it at $\tau_0 + 180^\circ$. This was implemented to quickly resolve a mismatch between how and where the intrinsic polarisation angle was calculated in the program and where the polarisation angles of the bands were calculated. The intrinsic polarisation angle was found between 0° and 180° , but the polarisation angles of the bands were found between -90° and 90° . The different ranges meant that if any of the polarisation angles were found to be between -90° and 0° , then the line of best fit would not align with the plotted polarisation angles. Plotting three lines seemingly resolved this issue because the different intercepts ensured that one of the three lines would always align with the polarisation angles in the plot. Figure 4.13b illustrates where we can see two lines, one matching the wrapped angles and one matching the unwrapped angles. Once the band polarisation angles were changed to be defined between 0° and 180° , the three lines were no longer needed, and the updated RM program now only plots one line. A late-stage version of the plot is shown in Figure 4.13c, where we see that there is only a single line of best fit.

x-axis of the peak pixel linear plot

The x -axis of the peak pixel linear plot did not originally correspond to the bands. The x -axis range also meant that the band A point was very close to the edge of the plot, and was partially cut off. The new x -ticks correspond to each band and are labelled accordingly, and the range was extended to completely show the band A point. Figure 4.13b shows the original axis, and Figure 4.13c shows the modified axis.

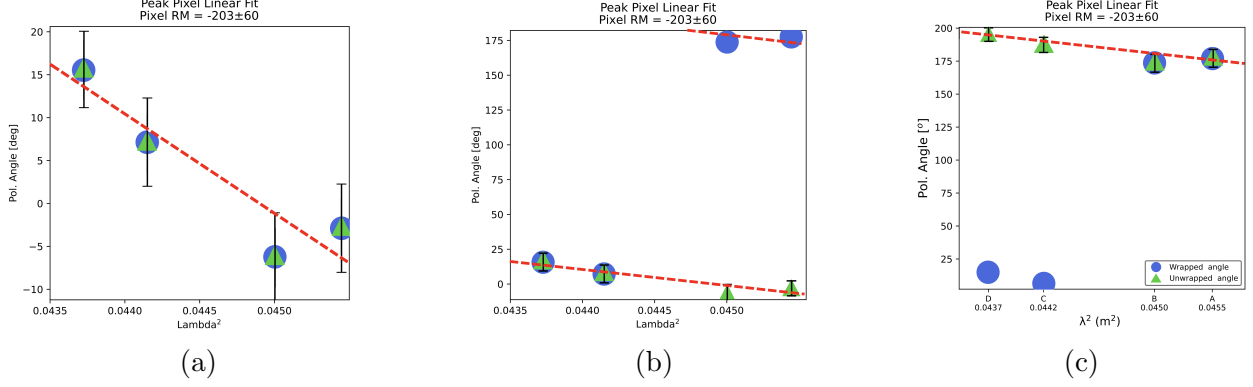


Figure 4.13: The Evolution of the Peak Pixel Linear Fit Plot.

These figures show the peak pixel linear plot at various stages in its evolution. Panel (a) shows the original plot produced by the Python program before any changes were made. Panel (b) shows the peak pixel plot after the wrapped angles were added. Panel (c) shows the late-stage version of the peak pixel plot.

Matching SI and PI

The original SI cutout and PI cutout were not centred at the same location. The SI cutout was originally centred one pixel above and to the right of the PI cutout. The x-axis ticks of the SI cutout also did not match the x -axis of the PI and RM plots. Both of these issues have been rectified.

Exporting FITS cutouts

The other major modification to the program is the option to export a FITS file of the SI cutout, PI cutout, and RM map to the output data folder for each pair. The FITS files allow the maps to be examined later without rerunning the program and to compare the RM maps directly with other FITS files.

Various other miscellaneous changes were also made to improve the readability of the plot panels, such as adding a super title to the panels indicating the source number and the mosaic being examined, moving the colorbar closer to the RM map to remove excess blank space, etc. The updated RM program is now available on GitHub (Tuttle et al., 2026).

In this chapter, I reviewed how my pair detection program works, how the RMs of the sources were calculated. This included a discussion of my addition of source masking to the program, and the other modifications that I made to the program. I will now present the pairs detected and the classification of each pair as physical, random, and unknown pair types.